

interpolate#

<https://pytorch.org/docs/stable/generated/torch.ao.nn.quantized.functional.interpolate.html>

Description:

Down/up samples the input to either the given size or the given scale_factor. See `torch.nn.functional.interpolate()` for implementation details. The input dimensions are interpreted in the form: mini-batch x channels x [optional depth] x [optional height] x width. The input quantization parameters propagate to the output.

Function Signature

```
class torch.ao.nn.quantized.functional.interpolate(input,  
size=None, scale_factor=None, mode='nearest',  
align_corners=None)[source]#
```

Parameters

Notes & Warnings

NOTE

Note The input quantization parameters propagate to the output.

NOTE

Note Only 2D/3D input is supported for quantized inputs

NOTE

Note Only the following modes are supported for the quantized inputs:
bilinear nearest

Detailed Documentation

Documentation

Down/up samples the input to either the given size or the given `scale_factor`

See `torch.nn.functional.interpolate()` for implementation details.

The input dimensions are interpreted in the form: mini-batch x channels x [optional depth] x [optional height] x width.

The input quantization parameters propagate to the output.

Only 2D/3D input is supported for quantized inputs

Only the following modes are supported for the quantized inputs:

`input` (Tensor) – the input tensor

`size` (int or Tuple[int] or Tuple[int, int] or Tuple[int, int, int]) – output spatial size.

`scale_factor` (float or Tuple[float]) – multiplier for spatial size. Has to match input size if it is a tuple.

`mode` (str) – algorithm used for upsampling: 'nearest' | 'bilinear'

`align_corners` (bool, optional) – Geometrically, we consider the pixels of the input and output as squares rather than points. If set to True, the input and output tensors are aligned by the center points of their corner pixels, preserving the values at the corner pixels. If set to False, the input and output tensors are aligned by the corner points of their corner pixels, and the interpolation uses edge value padding for out-of-boundary values, making this operation independent of input size when `scale_factor` is kept the same. This only has an effect when `mode` is 'bilinear'. Default: False